COMPUTER NETWORKS · SEMESTER 1 · WEEK 5

The application layer III in schemes

Electronic mail, Usenet, DNS and NAT — lecture, seminar and laboratories

Assoc. Prof. Nicolaie Popescu-Bodorin, PhD · 2026
+40722882471 · bodorin(on)live.com · https://lmrec.org/bodorin/

DNS (Domain Name System) · NAT (network address translation)

How this week is organised

A · Lecture 5
Mail, news, names, NAT

B · Seminar 5
Problems with solutions

C · Laboratory 5 · Linux
SMTP, POP3, dig, DNS in C, NAT

D · Laboratory 5 · Windows
PowerShell, Winsock, your NAT

Concepts

e-mail: SMTP, MIME, POP3, IMAP · Usenet · DNS · NAT

Calculations

SMTP and IMAP transcripts, base64 sizes, flooding, DNS timing and bytes, NAT tables

Tools on Linux

SMTP and POP3 by hand, dig, a DNS client in C, a NAT router in namespaces

Tools on Windows

the same mail servers, Resolve-DnsName, Winsock, finding your own NAT

Key idea This week completes the application layer: every protocol in the syllabus list is then covered.

SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · DNS (Domain Name System) · NAT (network address translation)

The application layer, complete

HTTP · HTTPS	Weeks 3–4
TELNET · SSH	Week 4
FTP	Week 4 (+ NAT this week)
SMTP · POP · IMAP	**this week**
Usenet	**this week**
DNS	**this week**
NAT	**this week**

this week's thread

mail needs DNS (MX records)

every protocol needs DNS

every home network needs NAT

— and NAT breaks some protocols

Key idea Mail, names and address translation: the infrastructure under every other application.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · SMTP (Simple Mail Transfer Protocol) · IMAP (Internet Message Access Protocol) · DNS (Domain Name System) · SSH (Secure Shell) · NAT (network address translation) · POP (Post Office Protocol) · FTP (File Transfer Protocol) · MX (mail exchanger)

Part A · Lecture 5: the application layer III

A · Lecture 5
Mail, news, names, NAT

→

B · Seminar 5
Problems with solutions

→

C · Laboratory 5 · Linux
SMTP, POP3, dig, DNS in C, NAT

→

D · Laboratory 5 · Windows
PowerShell, Winsock, your NAT

In this part

1 · The last week of the application layer

2 · Electronic mail: the system

3 · SMTP: pushing mail

4 · The message: format and MIME

5 · POP3, IMAP, webmail; protecting mail

6 · Usenet: news by flooding

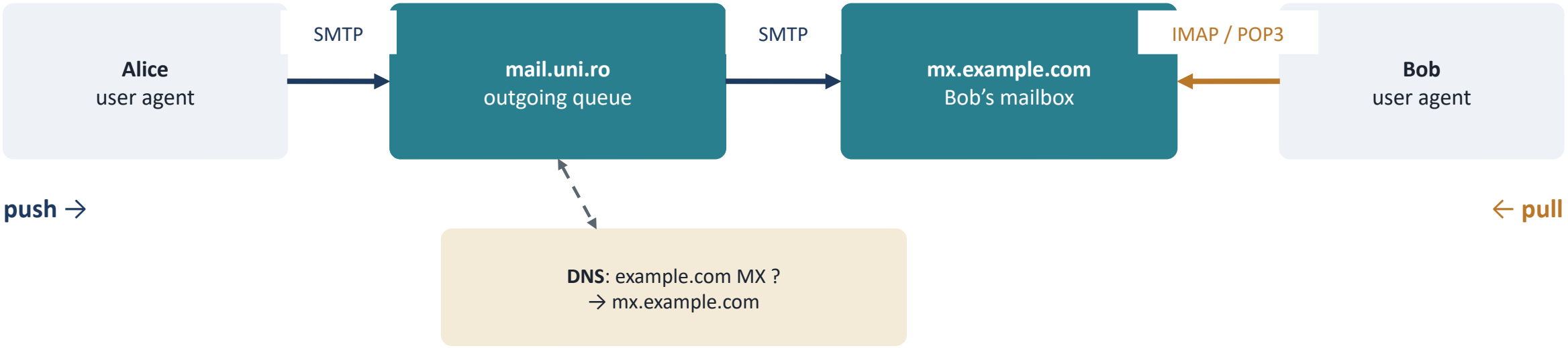
7 · DNS: services and hierarchy

8 · DNS: resolution, caching, records

9 · NAT: many hosts, one address

SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · DNS (Domain Name System) · NAT (network address translation)

2 · The e-mail system: agents, servers, protocols



1 **Asynchronous**: Bob need not be on line when Alice sends.

2 Receiving server down? The sender's server **queues** and retries for days, then bounces.

Key idea SMTP pushes between servers; IMAP or POP3 pull from the mailbox; DNS says where to push.

SMTP (Simple Mail Transfer Protocol) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · DNS (Domain Name System) · MX (mail exchanger)

3 · An SMTP dialogue

```
S: 220 mx.example.com ESMTP ready
C: EHLO mail.uni.ro
S: 250-mx.example.com
S: 250-SIZE 52428800
S: 250 STARTTLS
C: MAIL FROM:<alice@uni.ro>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: From: Alice <alice@uni.ro>
C: Subject: Lab 5
C:
C: Hello Bob, see you on Monday.
C: .
S: 250 OK: queued as 7F3A2
C: QUIT
S: 221 Bye
```



RFC 5321

2xx done
3xx send more
4xx try later
5xx permanent

550 no such user
552 too large

Key idea SMTP: greet, give the envelope, send the message, end it with a dot.

STARTTLS (start TLS on this connection) · ESMTP (Extended SMTP) · SMTP (Simple Mail Transfer Protocol) · EHLO (extended hello) · RCPT (recipient) · RFC (Request for Comments) · CR (carriage return) · LF (line feed)

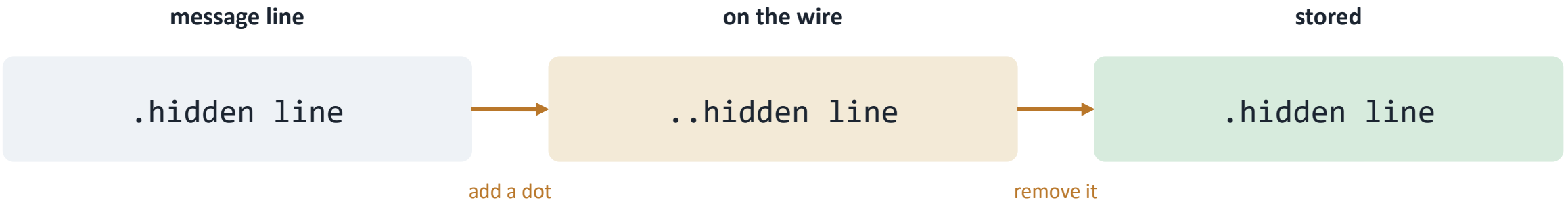
3 · The SMTP commands

Command	Meaning
`HELO / EHLO name`	greeting; EHLO asks the server to list its extensions: SIZE · 8BITMIME · STARTTLS · AUTH · SMTPUTF8
`MAIL FROM:<addr>`	start a transaction; the **reverse path**, where bounces go
`RCPT TO:<addr>`	one recipient — repeated for each
`DATA`	the message follows, ended by a line with a single dot
`RSET · NOOP · QUIT`	abort the transaction · do nothing · end the session
`STARTTLS · AUTH`	switch the connection to TLS (RFC 3207) · authenticate the user (RFC 4954)

Key idea MAIL FROM and RCPT TO are the envelope; everything after DATA is the message.

STARTTLS (start TLS on this connection) · SMTP (Simple Mail Transfer Protocol) · EHLO (extended hello) · RSET (reset) · TLS (Transport Layer Security) · RFC (Request for Comments)

3 · Where does the message end? Dot-stuffing



.
end of message

..
a line containing one dot — not the end

1 The delimiter method of Lecture 3, with escaping — like POP3's RETR later in this lecture.

Key idea A delimiter needs escaping: every data line that looks like the delimiter is changed on the wire.

POP3 (Post Office Protocol version 3) · RETR (retrieve)

3 · The envelope is not the header

envelope — used for delivery

```
MAIL FROM:<list-bounces@uni.ro>
RCPT TO:<bob@example.com>  RCPT TO:<dan@…>
```

letter inside — header, shown to the reader

```
From: Prof. Ionescu <ionescu@uni.ro>
To: Students <cn-2026@uni.ro>
```

1. **Bcc**: in the envelope, in no header line
2. **mailing list**: thousands of envelope addresses, To: names the list
3. **bounces**: go to MAIL FROM, not to From:
4. **forgery**: anyone can write any From: → SPF, DKIM, DMARC

Key idea Like paper mail: the post office reads the envelope, the reader reads the letterhead.

DMARC (Domain-based Message Authentication, Reporting and Conformance) · DKIM (DomainKeys Identified Mail) · RCPT (recipient) · SPF (Sender Policy Framework)

3 · Ports, encryption and the MX lookup

25	**server → server relay** STARTTLS if possible; ISPs block it from homes
587	**submission by user agents** AUTH, then STARTTLS (RFC 6409)
465	**submission over TLS** TLS from the first byte (RFC 8314)

```
$ dig +short example.com MX
10 mx1.example.com.
20 mx2.example.com.
```

mx1 (pref 10)
try first

mx2 (pref 20)
fallback

1. Lowest preference number first; if it does not answer, the next — and if none answers, queue and retry.

2. Our first application that **cannot work without DNS**.

Key idea Port 25 between servers, 587/465 from users; MX records name the servers that accept mail for a domain.

STARTTLS (start TLS on this connection) · TLS (Transport Layer Security) · DNS (Domain Name System) · RFC (Request for Comments) · MX (mail exchanger)

3 · SMTP and HTTP compared

	HTTP	SMTP
Direction	**pull**: the client asks for an object	**push**: the client delivers a message
Connection	persistent (HTTP/1.1)	persistent: several messages per connection
Encoding	any bytes (Content-Length)	7-bit ASCII lines; binary needs MIME or 8BITMIME
Several objects	one response per object	all parts in one multipart message
Codes	status codes	reply codes

Key idea HTTP pulls one object per response; SMTP pushes one message holding every part.

ASCII (American Standard Code for Information Interchange) · HTTP (HyperText Transfer Protocol) · SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions)

4 · The message: header and body (RFC 5322)

```
Received: from mail.uni.ro (…) by mx.example.com
    with ESMTPS; Mon, 26 Oct 2026 09:14:03 +0200
Received: from [192.168.1.34] (…) by mail.uni.ro
    with ESMTPSA; Mon, 26 Oct 2026 09:13:58 +0200
Message-ID: <a81f2@uni.ro>
Date: Mon, 26 Oct 2026 09:13:57 +0200
From: Alice <alice@uni.ro>
To: Bob <bob@example.com>
Subject: Lab 5

Hello Bob, see you on Monday.
```

trace
newest hop on top — read bottom-up

header fields
From · To · Cc · Subject · Date
Message-ID (unique) · Reply-To
In-Reply-To · References (threads)

empty line, then the **body**
plain RFC 5322 allows only ASCII text

Key idea The header is a list of Name: value lines; Received lines record the path, newest first.

ASCII (American Standard Code for Information Interchange) · RFC (Request for Comments)

4 · MIME: types, parts and encodings

Field	Meaning
`MIME-Version: 1.0`	the message uses MIME
`Content-Type`	text/plain; charset=utf-8 · image/jpeg · application/pdf · multipart/mixed; boundary=… — the media types HTTP uses too
`Content-Transfer-Encoding`	7bit · 8bit · quoted-printable (=C8) · **base64**
`Content-Disposition`	inline · attachment; filename="report.pdf"

Encoded words (RFC 2047) for non-ASCII header text:
Subject: =?utf-8?b?TGFib3JhdG9ydWwgNQ==?= → “Laboratorul 5”

Key idea MIME = labels (Content-Type) + packaging (multipart) + safe transport encodings (base64).

ASCII (American Standard Code for Information Interchange) · HTTP (HyperText Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · RFC (Request for Comments)

4 · A multipart message, part by part

```
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="==B7f2"

--==B7f2
Content-Type: text/plain; charset="utf-8"

Hello Bob, the report is attached.
--==B7f2
Content-Type: application/pdf
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="report.pdf"

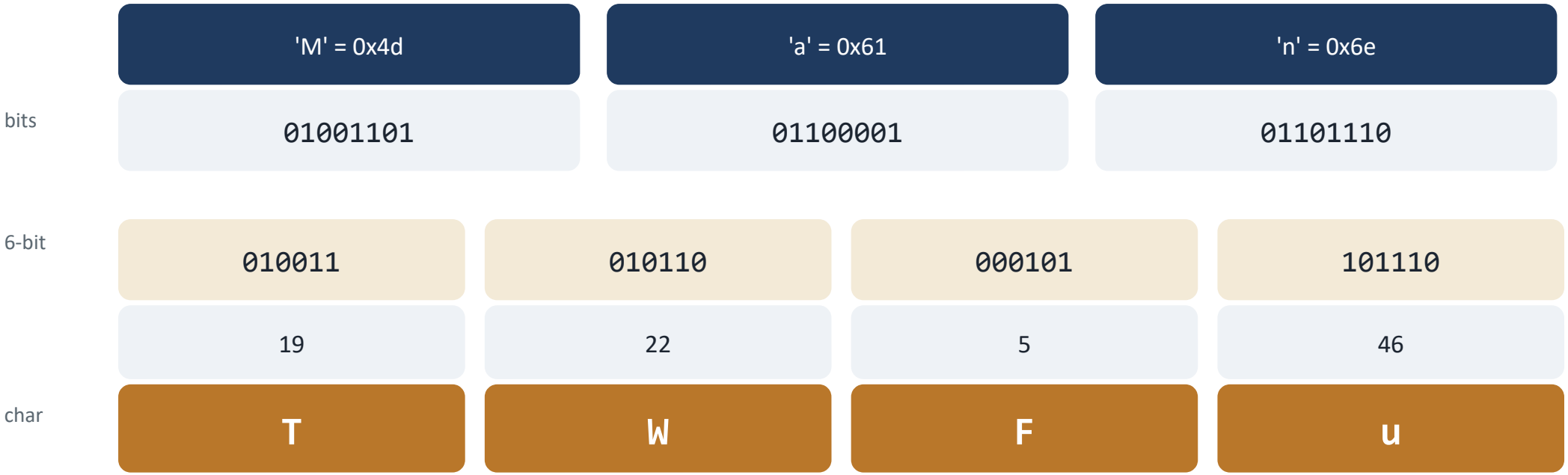
JVBERi0xLjcKJcfsj6IKNSAwIG9iago8PC9MZW5ndGgg…
--==B7f2--
```



Key idea multipart/mixed: one message, many parts, each labelled by its own Content-Type.

MIME (Multipurpose Internet Mail Extensions)

4 · base64: 3 bytes become 4 characters

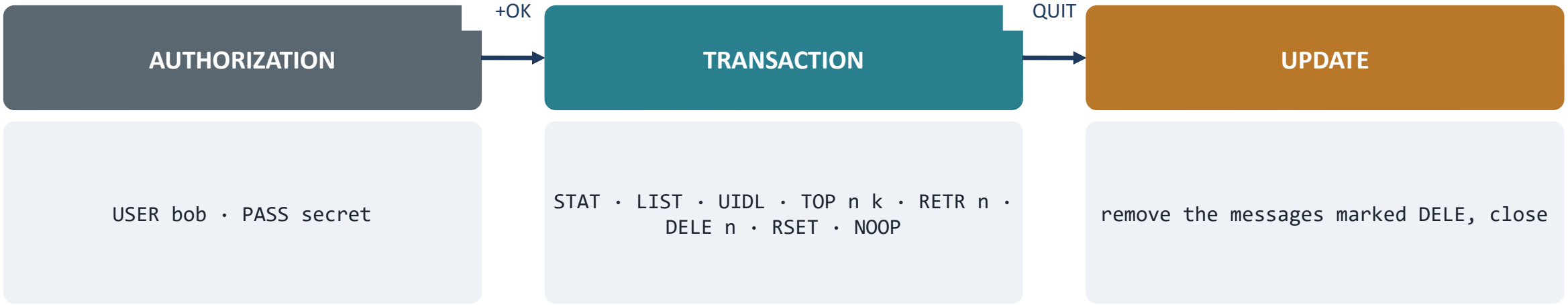


4/3 = +33 %, plus CR LF every 76 characters → about **+37 %**
a 10 MB attachment becomes ≈ 13.7 MB on the wire

Key idea “Man” → “TWFu”: binary made safe for 7-bit text, at the price of a third more bytes.

CR (carriage return) · LF (line feed)

5 · POP3: three states




```
C: STAT
S: +OK 2 4510
C: RETR 1
S: +OK 1240 octets
S: … the message …
S: .
```

1. Replies: **+OK** or **-ERR**; RETR ends with a dot (dot-stuffing).
2. Connection lost before QUIT → **nothing deleted**.

Key idea POP3 suits one device that downloads and keeps its mail; it knows no folders and no flags.

POP3 (Post Office Protocol version 3) · UIDL (unique-id listing) · STAT (status) · RETR (retrieve) · RSET (reset)

5 · IMAP: the mailbox stays on the server



1. Read on the phone → \Seen on the laptop too.
2. Fetch only headers, or one part of one message.
3. IDLE (RFC 2177): the server announces new mail at once instead of being polled.

Key idea IMAP is a remote mailbox; POP3 is a download service.

POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · RFC (Request for Comments)

5 · Reading an IMAP exchange: tags and literals

```
C: a1 LOGIN bob secret
S: a1 OK LOGIN completed
C: a2 SELECT INBOX
S: * 3 EXISTS
S: * 1 RECENT
S: a2 OK [READ-WRITE] SELECT completed
C: a3 FETCH 3 (FLAGS BODY.PEEK[HEADER.FIELDS (FROM SUBJECT)])
S: * 3 FETCH (FLAGS () BODY[HEADER.FIELDS …] {52}
S: From: Alice <alice@uni.ro>
S: Subject: Lab 5
S: )
S: a3 OK FETCH completed
C: a4 STORE 3 +FLAGS (\Seen)
C: a5 LOGOUT
```

`a1 … a5`

tags: several commands may be outstanding

`S: * 3 EXISTS`

untagged data lines

`{52}`

52 bytes of literal data follow — a length prefix inside a text protocol

`BODY.PEEK`

fetch without setting \Seen

Key idea Tags match replies to commands; literals carry exact byte counts — two framing tools in one protocol.

IMAP (Internet Message Access Protocol)

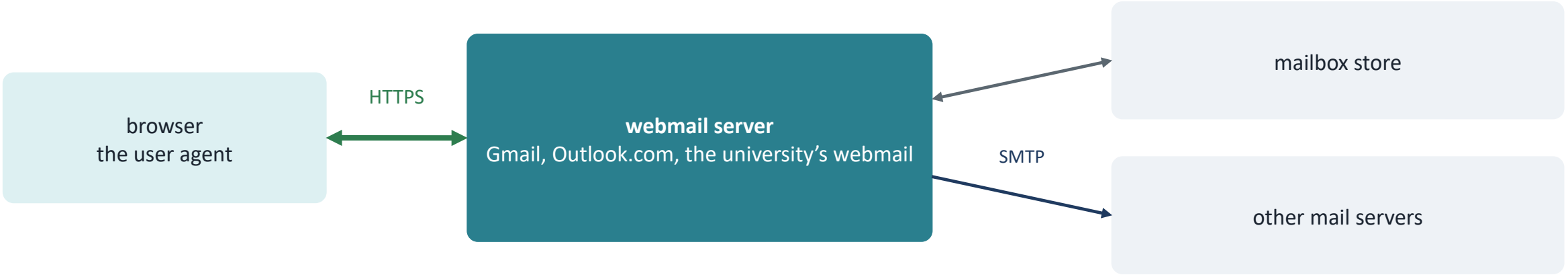
5 · POP3 or IMAP?

	POP3	IMAP
Mail lives	on the device (download)	on the server
Folders · flags	no	yes
Several devices	poorly: each downloads	same state everywhere
Slow link	whole messages (or TOP)	headers, parts, server search
Ports	110 · 995 (TLS)	143 · 993 (TLS)
Complexity	a dozen commands	a large protocol

Key idea One device, off line: POP3. Phone + laptop + web: IMAP.

POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · TLS (Transport Layer Security)

5 · Webmail and JMAP



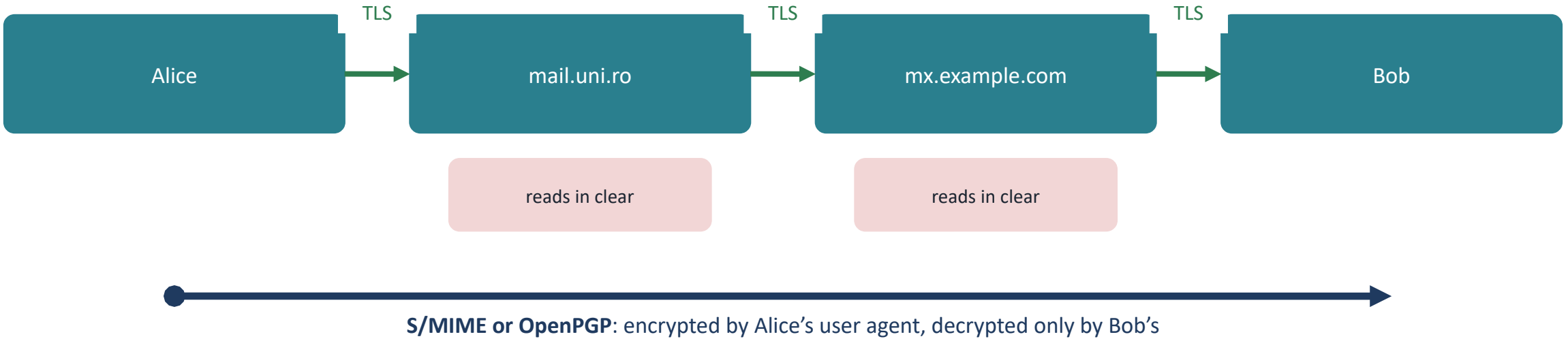
JMAP (RFCs 8620–8621): mailbox access as JSON over HTTPS — a modern replacement for IMAP, efficient on mobile devices

1 Many mobile apps use their providers' own protocols; standard clients still use IMAP and SMTP.

Key idea Webmail moves the user agent into the server; only HTTPS crosses the user's network.

HTTPS (HyperText Transfer Protocol Secure) · SMTP (Simple Mail Transfer Protocol) · IMAP (Internet Message Access Protocol) · JMAP (JSON Meta Application Protocol) · JSON (JavaScript Object Notation)

5 · Encryption: per hop, or end to end



1 Hop-by-hop TLS defeats the eavesdropper on a link; end-to-end encryption also defeats a curious or compromised server — but webmail and server-side search then cannot read the text either.

Key idea TLS protects the transport of mail; S/MIME and OpenPGP protect the mail itself.

MIME (Multipurpose Internet Mail Extensions) · TLS (Transport Layer Security)

5 · Against forged senders: SPF, DKIM, DMARC

SPF
RFC 7208

TXT: which addresses may send for the domain

```
v=spf1 ip4:193.231.0.0/16 include:_spf.google.com -all
```

DKIM
RFC 6376

a signature over header and body, key in DNS

```
DKIM-Signature: v=1; d=uni.ro; s=sel1; b=…
```

DMARC
RFC 7489

policy at _dmarc.domain: From: must align with SPF or DKIM

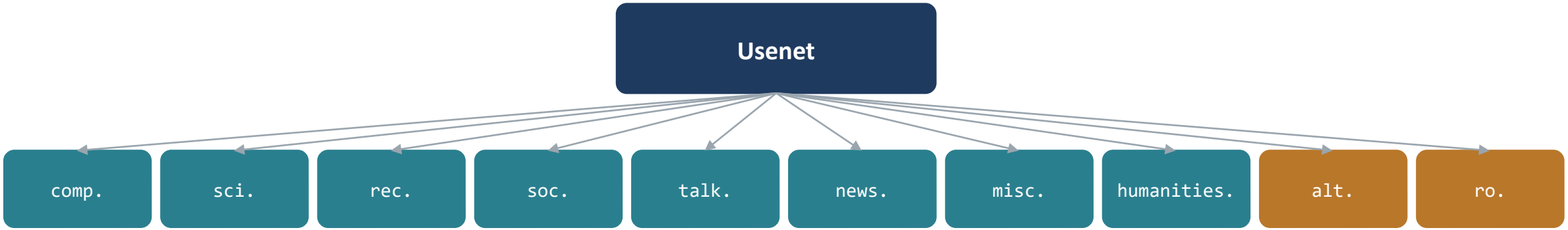
```
v=DMARC1; p=reject; rua=mailto:dmarc@uni.ro
```

Since 2024 the largest mailbox providers reject bulk mail that fails these checks; spam filters add content and reputation.

Key idea SPF: who may send; DKIM: was it signed; DMARC: what to do when they fail — all published in DNS.

DMARC (Domain-based Message Authentication, Reporting and Conformance) · DKIM (DomainKeys Identified Mail) · DNS (Domain Name System) · RFC (Request for Comments) · SPF (Sender Policy Framework) · TXT (text)

6 · Usenet: newsgroups in hierarchies



the “Big Eight” (teal) · alternative and national hierarchies (amber)

`comp.os.linux.networking`	`comp.protocols.tcp-ip`	`sci.physics`	`ro.comp.retele`

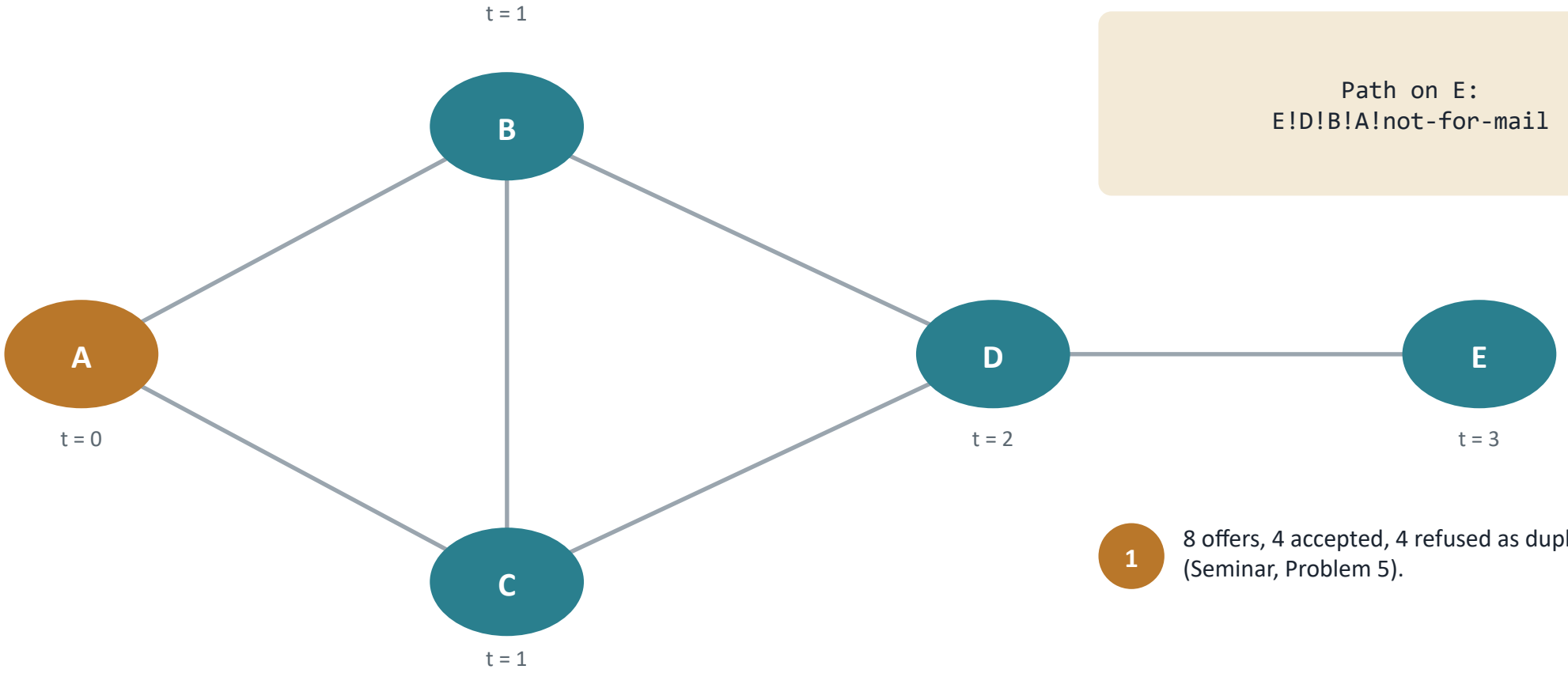
An article = a mail-like message (RFC 5536) + **Newsgroups:** · **Message-ID:** (unique) · **References:** (thread) · **Path:**

1991: Linux announced on comp.os.minix, the Web on alt.hypertext · 1994: the first commercial spam

Key idea Usenet is mail-shaped messages, addressed to topics instead of people.

RFC (Request for Comments)

6 · Flooding: how an article reaches every server




```
Path on E:
E!D!B!A!not-for-mail
```

1 8 offers, 4 accepted, 4 refused as duplicates (Seminar, Problem 5).

Key idea Duplicate suppression by Message-ID bounds the flood: each server accepts each article once.

6 · NNTP: reading and feeding news

```
C: GROUP comp.protocols.tcp-ip
S: 211 1234 3000 4233 comp.protocols.tcp-ip
C: OVER 4200-4233
C: ARTICLE 4233
C: POST
S: 340 send article, end with .
```

reader commands: LIST · GROUP · OVER · ARTICLE · HEAD · BODY · POST

```
C: IHAVE <a81f2@news.uni.ro>
S: 335 send it
S: 435 not wanted (already seen)

streaming feeds (RFC 4644):
CHECK · TAKETHIS — no round trip per article
```

server to server: “do you want it?” before sending

1 Discussion moved to mailing lists, forums and social networks — all client–server; flooding with duplicate suppression returns in **routing protocols**.

Key idea IHAVE asks before sending: a refusal costs one short line, not the article.

NNTP (Network News Transfer Protocol) · RFC (Request for Comments)

7 · DNS: what the naming system provides

A · AAAA	name → IPv4 / IPv6 address
CNAME	alias → canonical name
MX	mail servers of a domain
several A records	load distribution; CDNs answer by location
PTR	address → name (in-addr.arpa, ip6.arpa)
TXT · SRV · CAA	SPF, DMARC · service location · allowed CAs

Key idea DNS answers more than “which address?”: aliases, mail routing, load spreading, reverse lookups and policies.

DMARC (Domain-based Message Authentication, Reporting and Conformance) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · AAAA (IPv6 address record) · DNS (Domain Name System) · SPF (Sender Policy Framework) · PTR (pointer) · TXT (text) · SRV (service locator) · CAA (Certification Authority Authorization) · MX (mail exchanger)

7 · Why not one big server?

single point of failure
one outage stops the Internet

traffic
every lookup on the planet

distance
far from most users

maintenance
every change in the world, in one place

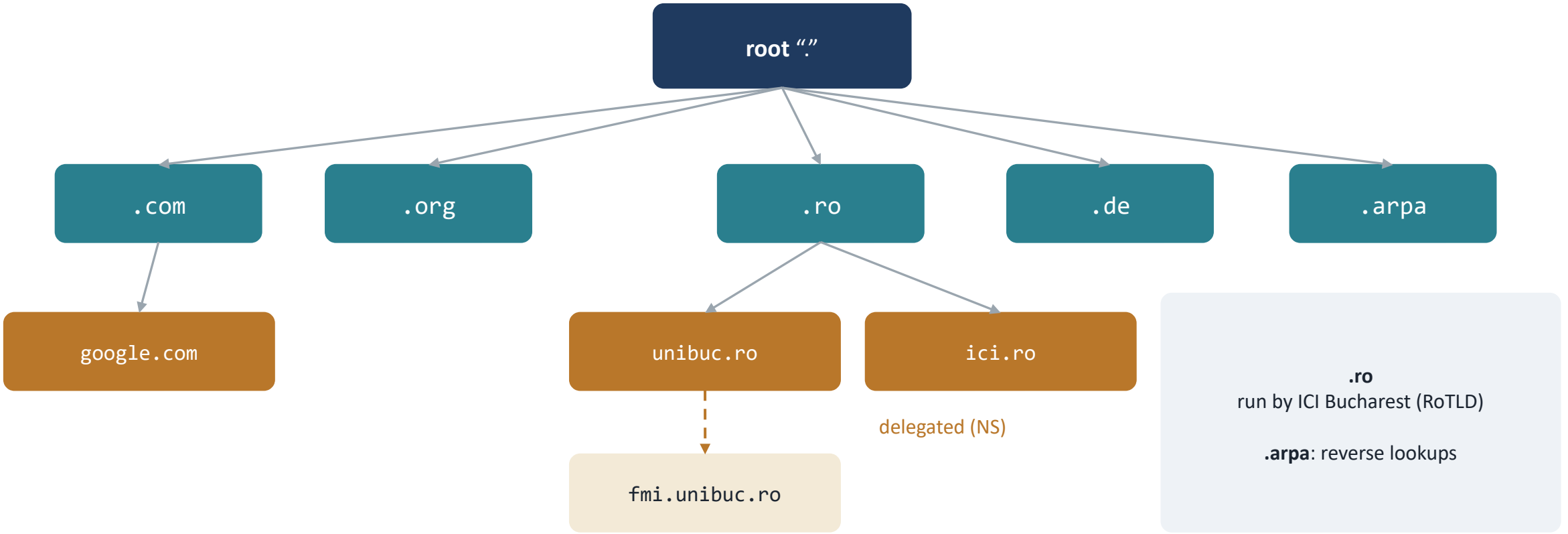
distributed (zones, delegation) · **hierarchical** (root → TLD → domain) · **replicated** (several servers per zone) · **cached** (TTL)

1 A core Internet function implemented at the application layer, at the edge — the end-to-end principle at work.

Key idea DNS scales because nobody holds the whole database and everybody caches.

DNS (Domain Name System) · TLD (top-level domain) · TTL (time to live)

7 · The hierarchy of zones

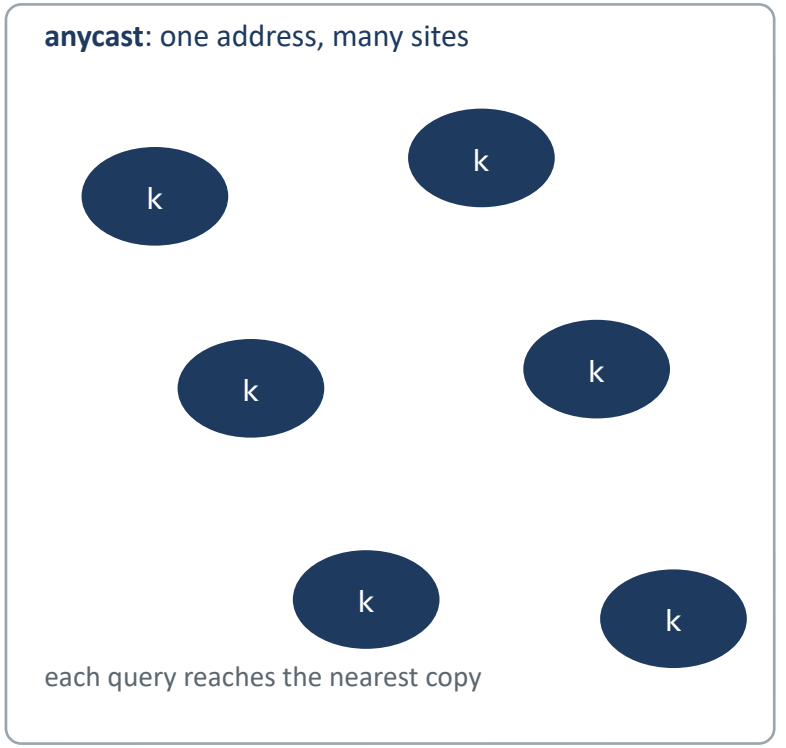


Key idea Each level knows only the next one down: delegation with NS records makes the tree.

RoTLD (the .ro registry) · NS (name server)

7 · Root servers and the four roles

stub resolver	a library in every host: sends queries
recursive resolver	ISP, university, 1.1.1.1, 8.8.8.8, 9.9.9.9: does the work, caches
root / TLD servers	answer with referrals one level down
authoritative server	holds a zone’s records; answers with AA set



Key idea Hosts ask resolvers; resolvers ask the hierarchy; only authoritative servers know the answer firsthand.

ISP (Internet service provider) · TLD (top-level domain) · AA (authoritative answer)

8 · Recursive and iterative resolution

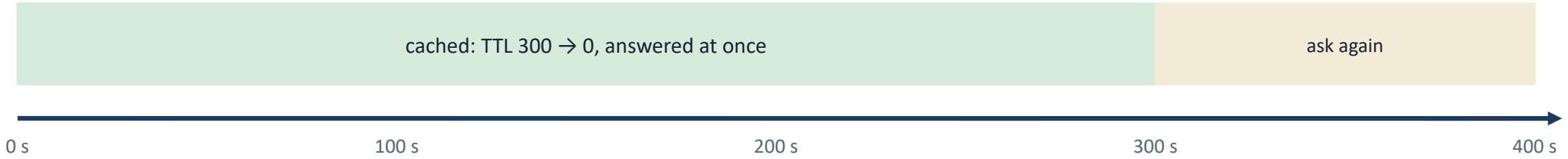


1 Every answer is **cached** by the resolver.

Key idea One recursive question from the host, a chain of iterative questions from the resolver.

TLD (top-level domain) · AA (authoritative answer)

8 · Caching and TTL



dig @1.1.1.1 www.google.com
TTL counts down: 300, 287, …

root and TLD referrals
cached for a day or two

negative answers (NXDOMAIN)
cached too: SOA's last number

moving a service?
lower the TTL a day before

Key idea TTL trades speed for freshness: long TTLs mean fast, cheap lookups and slow changes.

SOA (start of authority) · TLD (top-level domain) · TTL (time to live)

8 · Resource records and a zone file

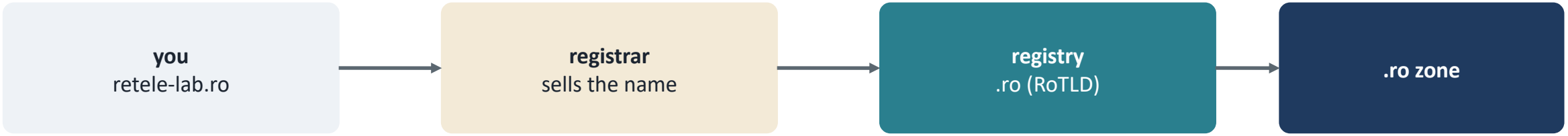
```
$ORIGIN example.ro.
@     3600   IN SOA   ns1 admin ( 2026102601
                      7200 900 1209600 300 )
@     86400  IN NS    ns1.example.ro.
@     86400  IN NS    ns2.provider.net.
ns1   86400  IN A     192.0.2.53
@     3600   IN MX    10 mail.example.ro.
mail  3600   IN A     192.0.2.25
www   300    IN A     192.0.2.80
www   300    IN AAAA  2001:db8::80
shop  3600   IN CNAME shops.cdn-provider.net.
@     3600   IN TXT   "v=spf1 mx -all"
```

SOA	primary, admin, serial, timers, negative TTL
NS	the zone's servers (delegation)
A / AAAA	addresses
MX	mail server + preference
CNAME	alias
TXT	free text: SPF, DMARC

Key idea @ is the zone's own name; a name without a final dot is relative to $ORIGIN.

DMARC (Domain-based Message Authentication, Reporting and Conformance) · CNAME (canonical name) · AAAA (IPv6 address record) · SPF (Sender Policy Framework) · SOA (start of authority) · TTL (time to live) · MX (mail exchanger) · NS (name server) · IN (Internet class)

8 · Registrar, registry and glue




```
retele-lab.ro.      NS  ns1.retele-lab.ro.
retele-lab.ro.      NS  ns.host.net.
ns1.retele-lab.ro.  A   193.0.2.1     ← glue
```

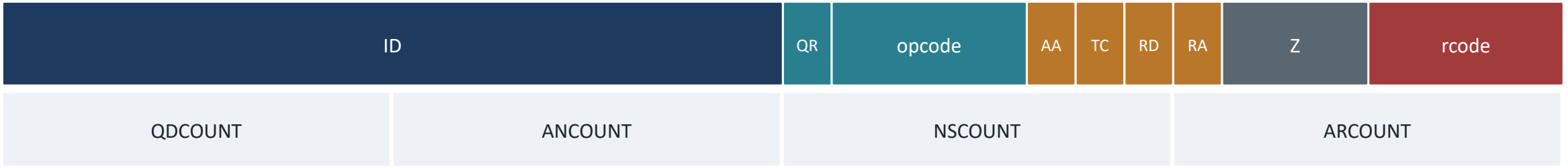
1 Without glue, finding ns1 would need ns1 itself — a circle.

2 ns.host.net needs no glue: it is found through .net. The apex (@) cannot be a CNAME — it already has NS and MX records.

Key idea Delegation = NS records in the parent zone, plus glue when the server lives inside the child zone.

CNAME (canonical name) · RoTLD (the .ro registry) · MX (mail exchanger) · NS (name server)

8 · The DNS message: a 12-byte header and four sections



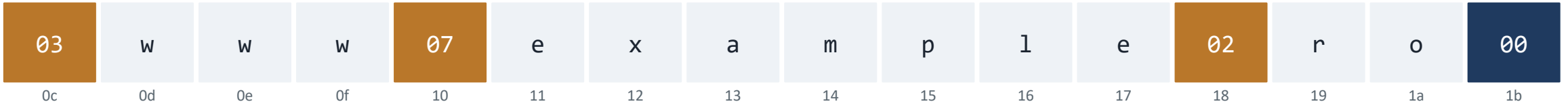
Question	name as labels · QTYPE · QCLASS
Answer	the records asked for
Authority	NS records of a referral
Additional	extra records: glue addresses

rcode
0 no error
2 server failure
3 no such name (NXDOMAIN)
5 refused

Key idea QR says query or reply; AA authoritative; TC truncated; RD asks for recursion; RA offers it.

DNS (Domain Name System) · NS (name server) · QR (query/response) · AA (authoritative answer) · RD (recursion desired) · RA (recursion available)

8 · Names as labels, and compression pointers



the question name at offset 0x0c — www.example.ro

`c0 0c` → offset 12: “www.example.ro”

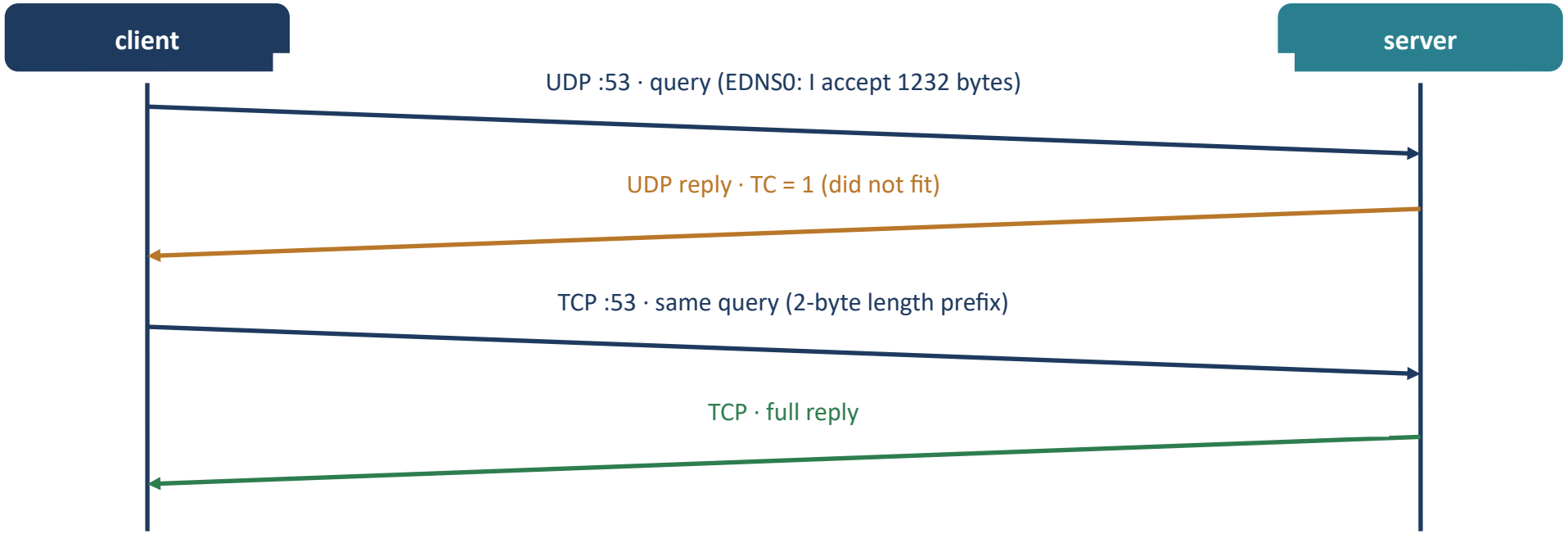
`03 w e b c0 10` → “web” + offset 16: “web.example.ro”

1100 0000 0000 1100
11 = pointer
00 0000 0000 1100 = 12

1 A careful decoder limits the number of jumps: a malicious pointer loop must not hang it (Lab 5, read_name).

Key idea Labels make names parseable; pointers make replies small.

8 · UDP first, TCP when needed



1 Originally UDP replies ≤ 512 bytes.

2 TCP also for zone transfers.

Key idea UDP for speed, TCP for size: the TC bit tells the client to switch.

TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TC (truncated)

8 · Attacks and defences: poisoning, DNSSEC, DoT, DoH

cache poisoning
a forged reply with the right ID and port arrives first →
all the resolver's users go to a false address
(Kaminsky, 2008; defence: random source ports)

DNSSEC
signatures (RRSIG), keys (DNSKEY), a chain of DS records up to the root
→ **authenticity**, not secrecy

DoT (port 853) · **DoH** (RFC 8484, HTTPS)
encrypt host ↔ resolver: the café Wi-Fi no longer sees your lookups

DNS is a target too
2016: the Mirai botnet against the DNS provider Dyn — major sites unreachable for hours

Key idea DNSSEC proves an answer is genuine; DoT and DoH hide the question — different guarantees.

DNSSEC (DNS Security Extensions) · DNSKEY (DNS public key) · HTTPS (HyperText Transfer Protocol Secure) · RRSIG (resource record signature) · DNS (Domain Name System) · RFC (Request for Comments) · DoH (DNS over HTTPS) · DoT (DNS over TLS)

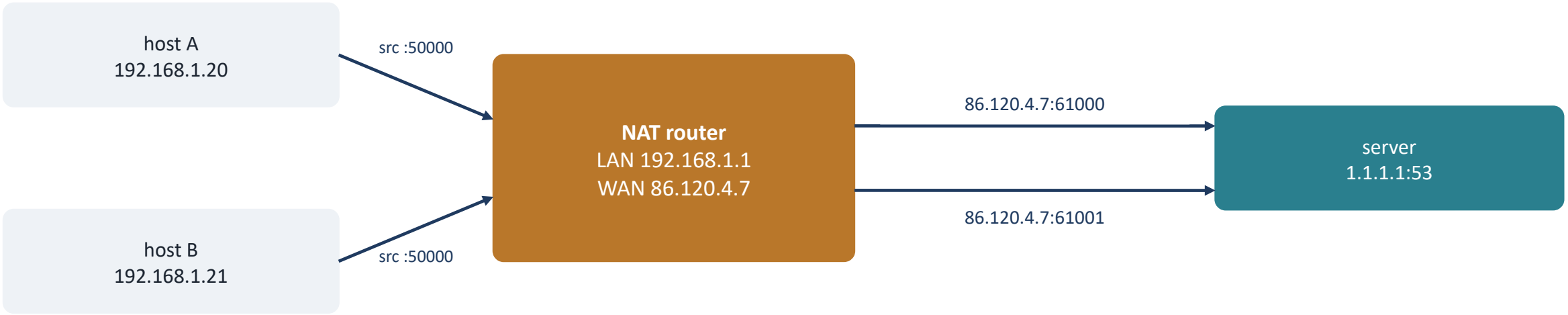
9 · IPv4 ran out: private addresses

`10.0.0.0/8`	16.7 million addresses
`172.16.0.0/12`	1 million addresses
`192.168.0.0/16`	65 536 addresses — most homes
`100.64.0.0/10`	shared space for carrier-grade NAT (RFC 6598)

Key idea Millions of networks reuse the same private addresses — so they need translation at the edge.

IPv4 (Internet Protocol version 4) · RFC (Request for Comments) · NAT (network address translation)

9 · NAT: the router rewrites addresses and ports



inside	outside	remote
192.168.1.20:50000	86.120.4.7:61000	1.1.1.1:53 UDP
192.168.1.21:50000	86.120.4.7:61001	1.1.1.1:53 UDP

Key idea Outgoing: rewrite the source and remember; incoming: look up the port and rewrite the destination back.

UDP (User Datagram Protocol) · NAT (network address translation) · LAN (local area network) · WAN (wide area network)

9 · One packet out, one reply back

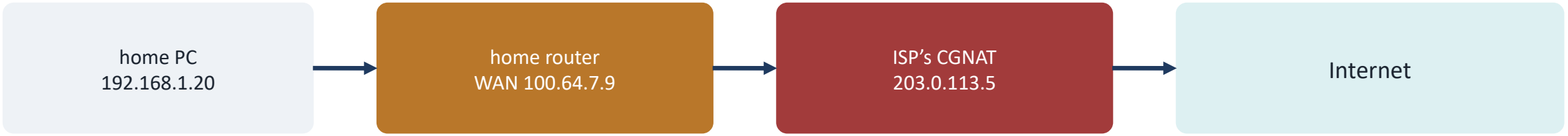
inside → router	`192.168.1.20:50000 → 1.1.1.1:53`	rewrite source, create mapping
router → Internet	`86.120.4.7:61000 → 1.1.1.1:53`	
Internet → router	`1.1.1.1:53 → 86.120.4.7:61000`	look up 61000, rewrite destination
router → inside	`1.1.1.1:53 → 192.168.1.20:50000`	

1 Ports are translated too: properly NAPT — network address and port translation (RFC 3022).

Key idea A reply finds its way home only because the outgoing packet created a mapping first.

RFC (Request for Comments)

9 · How many hosts fit behind one address? CGNAT



two translations on the path

capacity

64 000 ports / 2 000 per customer = 32 customers per address

40 000 customers → 1 250 public addresses

consequences

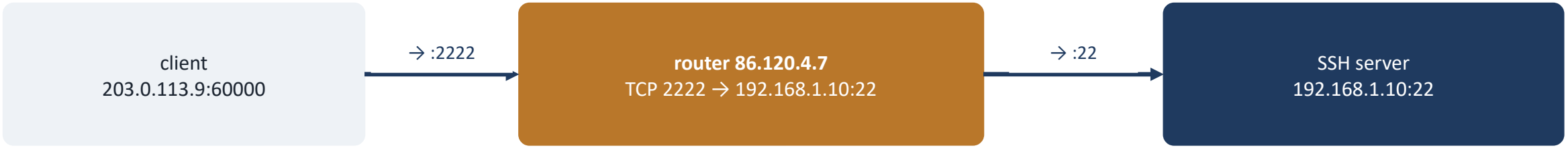
no port forwarding possible for the customer

shared address: blocking one abuser blocks many

Key idea Carrier-grade NAT stretches IPv4 further — and takes control of incoming connections away from the user.

IPv4 (Internet Protocol version 4) · NAT (network address translation) · ISP (Internet service provider) · WAN (wide area network)

9 · Inbound connections: port forwarding



1. **static**: configured by hand on the router (DNAT)

2. **UPnP IGD · PCP (RFC 6887)**: the application asks the router for a mapping

3. **no mapping**: packet dropped (or RST): NAT blocks unsolicited traffic — but is no firewall

Key idea Outgoing connections create mappings automatically; incoming ones need someone to create them first.

UPnP (Universal Plug and Play) · TCP (Transmission Control Protocol) · SSH (Secure Shell) · RFC (Request for Comments) · RST (reset) · NAT (network address translation) · PCP (Port Control Protocol) · IGD (Internet Gateway Device)

9 · What NAT breaks

mappings expire

idle UDP after tens of seconds, TCP after minutes or hours → keepalives

addresses inside messages

active FTP's PORT, SIP telephony → an ALG rewrites them — impossible if encrypted

peer-to-peer

two hosts both behind NAT cannot simply connect

end-to-end principle

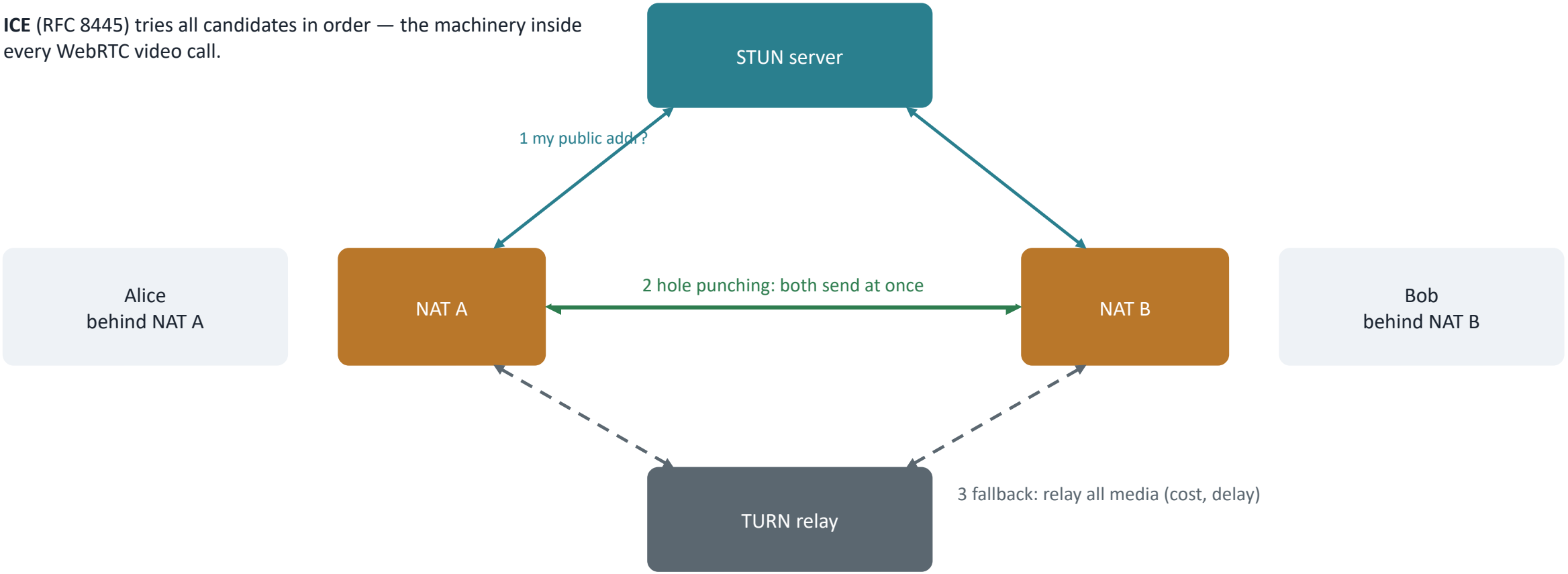
a router edits transport headers; a host's address is not what peers see

Key idea NAT kept IPv4 alive for thirty years at the price of the Internet's original end-to-end design.

IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · NAT (network address translation) · SIP (Session Initiation Protocol) · FTP (File Transfer Protocol) · ALG (application-level gateway)

9 · NAT traversal: STUN, hole punching, TURN, ICE

ICE (RFC 8445) tries all candidates in order — the machinery inside every WebRTC video call.



Key idea Each outgoing packet opens a door in its own NAT; STUN tells each peer which door the other must knock on.

WebRTC (Web Real-Time Communication) · STUN (Session Traversal Utilities for NAT) · TURN (Traversal Using Relays around NAT) · RFC (Request for Comments) · NAT (network address translation) · ICE (Interactive Connectivity Establishment)

9 · IPv6 makes NAT unnecessary

IPv4 + NAT	IPv6
private addresses inside	a global address per device
one shared public address	no translation
translation tables, ALGs, STUN/TURN	end-to-end restored
inbound needs forwarding	a stateful firewall decides what comes in

Key idea NAT is a workaround for too few addresses; we return to it, and to IPv6, with the network layer.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · STUN (Session Traversal Utilities for NAT) · TURN (Traversal Using Relays around NAT) · NAT (network address translation)

Lecture 5 summary

E-mail system
user agents, servers, queues; push and pull

NAT
mappings, CGNAT, forwarding, traversal

SMTP
envelope vs header; dot-stuffing; 25/587/465

DNS
hierarchy, resolution, TTL, records, messages

Week 5

MIME
types, multipart, base64 (+37 %)

Usenet
newsgroups, flooding, Message-ID, NNTP

POP3 · IMAP
download vs remote mailbox; tags

Protecting mail
TLS per hop, S/MIME, SPF, DKIM, DMARC

Key idea With mail, news, names and translation, the application layer is complete: next, the transport layer.

DMARC (Domain-based Message Authentication, Reporting and Conformance) · CGNAT (carrier-grade NAT) · SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · NNTP (Network News Transfer Protocol) · DKIM (DomainKeys Identified Mail) · TLS (Transport Layer Security) · DNS (Domain Name System) · NAT (network address translation) · SPF (Sender Policy Framework) · TTL (time to live)

Part B · Seminar 5: problems with solutions

A · Lecture 5
Mail, news, names, NAT

→

B · Seminar 5
Problems with solutions

→

C · Laboratory 5 · Linux
SMTP, POP3, dig, DNS in C, NAT

→

D · Laboratory 5 · Windows
PowerShell, Winsock, your NAT

In this part

Problems 1–2 · An SMTP dialogue; size and path

Problem 5 · Flooding news

Problems 9–11 · NAT tables, capacity, traversal

Problems 3–4 · POP3 or IMAP; reading IMAP

Problems 6–8 · DNS timing, bytes, a zone

Homework

SMTP (Simple Mail Transfer Protocol) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · DNS (Domain Name System) · NAT (network address translation)

Problem 1 · A mailing-list delivery, read from the log

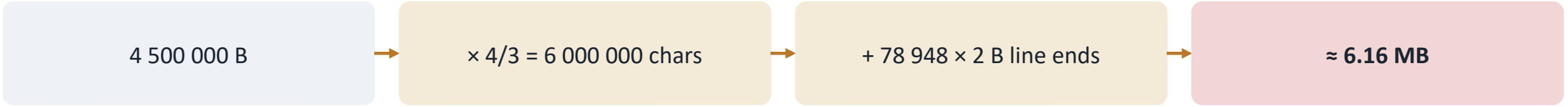
```
C: MAIL FROM:<list-bounces@uni.ro>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: RCPT TO:<carol@example.com>
S: 550 5.1.1 No such user
C: RCPT TO:<dan@example.com>
S: 250 OK
C: DATA
S: 354 Go ahead
C: From: Prof. Ionescu <ionescu@uni.ro>
C: To: Students <cn-2026@uni.ro>
C: …
C: ..and bring your student card.
C: .
S: 250 OK queued as 91BC
```

1. Delivered to **Bob and Dan**. Carol's 550 → the sending server bounces to **list-bounces@uni.ro**, not to the professor.
2. Nobody is in To: — delivery uses only the **envelope**: mailing lists and Bcc work this way.
3. Bob reads `.and bring your student card.` — one dot removed.
4. Missing: **STARTTLS** after EHLO (the server offered it). Without it the whole dialogue is readable on the path.

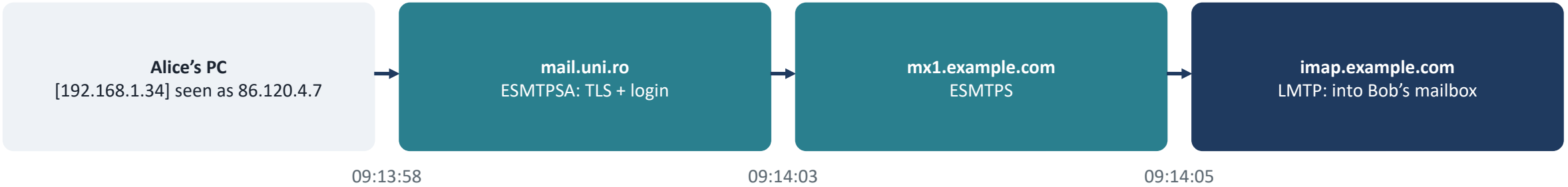
Key idea Read the envelope for delivery and bounces, the header only for display.

STARTTLS (start TLS on this connection) · EHLO (extended hello) · RCPT (recipient)

Problem 2 · Size and path of a message



below the server's SIZE limit of 26 214 400 bytes (25 MiB): accepted



1 Longest hop: mail.uni.ro → mx1 (5 s: queue, checks). Clocks differ slightly: indicative only.

2 Private address on her PC, another address seen by the server: Alice is **behind NAT**.

Key idea Received fields are a travel log written by each server — newest at the top.

LMTP (Local Mail Transfer Protocol) · TLS (Transport Layer Security) · NAT (network address translation)

Problem 3 · POP3 or IMAP?

researcher on a ship, 1 h link a day, one laptop	**POP3**	download all in the window, read off line
manager: phone, tablet, office PC, folders	**IMAP**	folders and flags on the server, same everywhere
shop order robot, every minute, each order once	**POP3**	UIDL to skip seen messages, then DELE
student: 3 000 subjects over a slow link, then 2 messages	**IMAP**	FETCH subjects only, then two messages

Key idea POP3 for one device and simple robots; IMAP whenever state must be shared or the link is slow.

POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · UIDL (unique-id listing) · DELE (delete)

Problem 4 · Reading an IMAP session

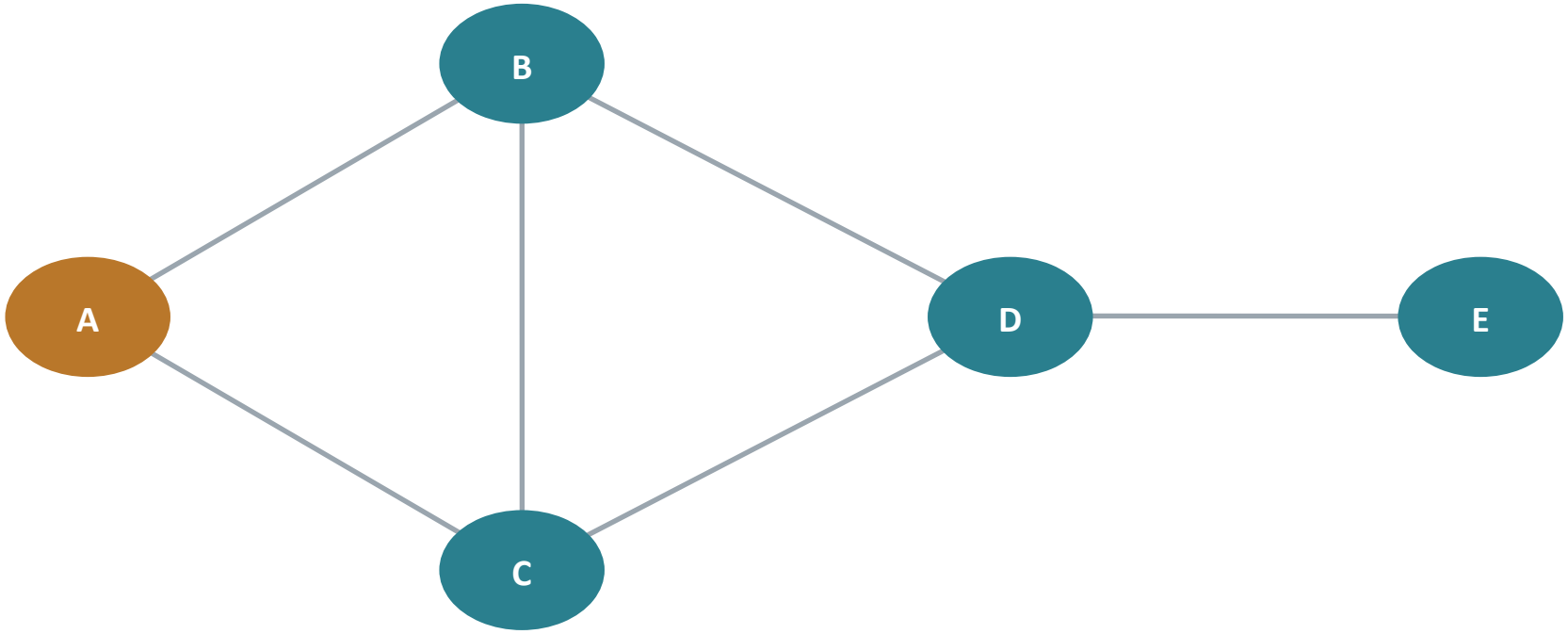
```
C: t1 SELECT INBOX
S: * 12 EXISTS
S: t1 OK [READ-WRITE] SELECT completed
C: t2 SEARCH UNSEEN FROM "ionescu"
S: * SEARCH 9 12
S: t2 OK SEARCH completed
C: t3 FETCH 12 (RFC822.SIZE BODY.PEEK[…(SUBJECT)])
S: * 12 FETCH (RFC822.SIZE 6158210 … {22}
S: Subject: Exam room
S: )
S: t3 OK FETCH completed
C: t4 STORE 9 +FLAGS (\Deleted)
C: t5 EXPUNGE
```

1. 12 messages; 9 and 12 unread from Ionescu — searched **on the server**.
2. PEEK keeps the message unread; {22} = 22 bytes of literal data.
3. 6 MB: fetch BODYSTRUCTURE and the text part, not the attachment.
4. t4 only marks \Deleted; t5 EXPUNGE removes it — like POP3's UPDATE state.

Key idea IMAP lets the client ask small questions about a large mailbox.

POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol)

Problem 5 · Flooding an article through five servers



offer	result
A→B	accepted, t = 1
A→C	accepted, t = 1
B→C	refused: seen
C→B	refused: seen
B→D	accepted, t = 2
C→D	refused: seen
D→C	refused: seen
D→E	accepted, t = 3

E stores Path: E!D!B!A!not-for-mail · 8 offers: 4 accepted, 4 refused

Key idea Path avoids sending back along the route; Message-ID stops every other duplicate.

Problem 6 · How long does a DNS lookup take?



the thin grey bar: laptop ↔ resolver, 2 ms · then the resolver's queries, in milliseconds

cold cache: **152 ms**

30 s later: everything cached
2 ms

5 min later: A expired (TTL 60), CNAME still cached
2 + 40 = 42 ms

1 Why TTL 60 for the CDN's A record? To steer users quickly to a nearer or healthier server — at the cost of more queries.

Key idea A CNAME into another TLD costs a second walk down the tree on a cold cache.

CNAME (canonical name) · DNS (Domain Name System) · CDN (content distribution network) · TLD (top-level domain) · TTL (time to live)

Problem 7 · Decoding a DNS reply byte by byte

```
0000  1a 2b 81 80 00 01 00 02 00 00 00 00 03 77 77 77
0010  07 65 78 61 6d 70 6c 65 02 72 6f 00 00 01 00 01
0020  c0 0c 00 05 00 01 00 00 01 2c 00 06 03 77 65 62
0030  c0 10 c0 2c 00 01 00 01 00 00 01 2c 00 04 c0 00
0040  02 50
```

compression saved 38 bytes
104 → 66 bytes

header	ID 1a2b · flags 8180: QR=1, RD=1, RA=1, AA=0, rcode 0 · QD 1, AN 2
question	www.example.ro · A · IN
answer 1	c0 0c → www.example.ro · **CNAME** · TTL 300 · web + c0 10 → **web.example.ro**
answer 2	c0 2c → web.example.ro · **A** · TTL 300 · c0 00 02 50 = **192.0.2.80**

Key idea Read a DNS message with three tools: the 12-byte header, length-prefixed labels, and pointers (c0 xx).

CNAME (canonical name) · DNS (Domain Name System) · TTL (time to live) · QR (query/response) · AA (authoritative answer) · RD (recursion desired) · RA (recursion available) · IN (Internet class)

Problem 8 · Designing a small zone

```
@     86400 IN NS    ns1.retele-lab.ro.
@     86400 IN NS    ns.host.net.
ns1   86400 IN A     193.0.2.1
@     3600  IN A     193.0.2.80
www   3600  IN CNAME retele-lab.ro.
@     3600  IN MX    10 mail.retele-lab.ro.
@     3600  IN MX    20 mx.host.net.
mail  3600  IN A     193.0.2.25
@     3600  IN TXT   "v=spf1 ip4:193.0.2.25 -all"
```

1. Apex: an A record — it cannot be a CNAME (it has NS and MX).
2. .ro publishes both NS records + **glue** for ns1.
3. Move next Friday: lower the A TTL to 60 s on Thursday.

SPF -all: every other sender fails the check

Key idea A zone is a short text file — but each line is a promise the rest of the Internet will rely on.

CNAME (canonical name) · SPF (Sender Policy Framework) · TTL (time to live) · MX (mail exchanger) · NS (name server) · IN (Internet class)

Problem 9 · Filling a NAT table

#	packet	after translation
1	192.168.1.10:51000 → 142.250.1.1:443 TCP	86.120.4.7:51000 → 142.250.1.1:443
2	192.168.1.11:51000 → 142.250.1.1:443 TCP	86.120.4.7:40000 (51000 taken)
3	192.168.1.11:5353 → 1.1.1.1:53 UDP	86.120.4.7:5353 (separate UDP table)
4	142.250.1.1:443 → 86.120.4.7:40000	→ 192.168.1.11:51000
5	203.0.113.9:22 → 86.120.4.7:22 SYN	no mapping: dropped (or RST)

port forward TCP 2222 → 192.168.1.10:22

outside: 203.0.113.9:60000 → 86.120.4.7:2222 · inside: 203.0.113.9:60000 → 192.168.1.10:22

Key idea Outgoing packets write the table; incoming packets can only read it.

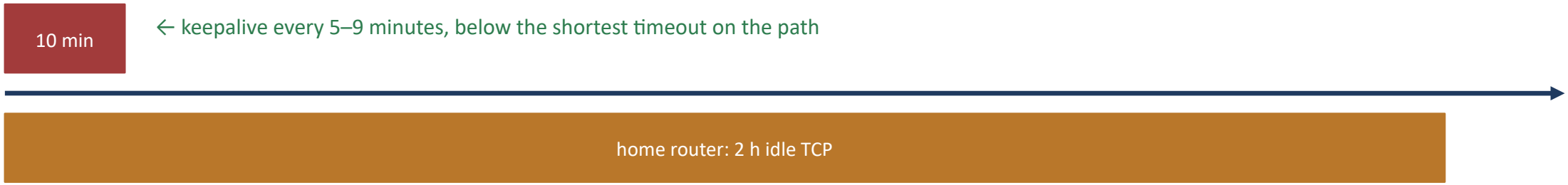
TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · SYN (synchronize) · RST (reset) · NAT (network address translation)

Problem 10 · CGNAT capacity and keepalives

2 048 × 64 000
= 131 million ports

÷ 100 000 customers
≈ 1 310 ports each

a family
browsers (20–50 connections a page), phones, games: usually enough



1 A CGNAT customer cannot forward ports at all: the mapping lives on the ISP's NAT. Options: a public IPv4 address, IPv6, or a relay.

Key idea The shortest timeout on the path sets the keepalive interval for every long-lived connection.

CGNAT (carrier-grade NAT) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · NAT (network address translation) · ISP (Internet service provider)

Problem 11 · A video call between two NATs

1 · STUN	each learns its public address and port
2 · signalling	they exchange them through the app’s server
3 · hole punching	both send to each other at once: each outgoing packet opens its own NAT
4 · TURN	symmetric NAT or UDP blocked → a relay carries all media

works when the NATs are **endpoint-independent** (RFC 4787): same public port whatever the destination

Key idea ICE tries direct candidates first and falls back to TURN: delay and the relay’s bandwidth are the price.

STUN (Session Traversal Utilities for NAT) · TURN (Traversal Using Relays around NAT) · UDP (User Datagram Protocol) · RFC (Request for Comments) · NAT (network address translation) · ICE (Interactive Connectivity Establishment)

Homework

1. In your own mailbox (Gmail: ⋮ → Show original) copy the Received fields and Authentication-Results; draw the path and who checked SPF, DKIM, DMARC.

2. With `dig +norecurse`, resolve www.unibuc.ro by hand from a.root-servers.net, following each referral.

3. Write in hexadecimal the full query for unibuc.ro MX, ID 0x0042, RD set; check it with tcpdump in Laboratory 5.

4. In half a page: why a NAT router must rewrite active FTP's PORT, why it cannot under FTPS, and what passive mode changes.

Key idea Each task turns a scheme from this week into something you can check on a real network.

DMARC (Domain-based Message Authentication, Reporting and Conformance) · DKIM (DomainKeys Identified Mail) · NAT (network address translation) · SPF (Sender Policy Framework) · FTP (File Transfer Protocol) · MX (mail exchanger) · RD (recursion desired)

Part C · Laboratory 5 on Linux: mail, DNS, NAT

A · Lecture 5
Mail, news, names, NAT

→

B · Seminar 5
Problems with solutions

→

C · Laboratory 5 · Linux
SMTP, POP3, dig, DNS in C, NAT

→

D · Laboratory 5 · Windows
PowerShell, Winsock, your NAT

In this part

Setup · servers, files, tools

Task 1 · SMTP by hand

Task 2 · MIME on the wire

Task 3 · POP3 by hand, an IMAP greeting

Task 4 · DNS with dig

Task 5 · A DNS client in C over UDP

Task 6 · A NAT router of your own; STUN

SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · STUN (Session Traversal Utilities for NAT) · UDP (User Datagram Protocol) · DNS (Domain Name System) · NAT (network address translation)

Setup · Your own mail system on one machine



File	Role
`python/talk.py host port [--tls]`	lines out with CR LF, every received line printed after <-
`python/send_mail.py`	a MIME message with an attachment, SMTP dialogue shown
`linux/dns_query.c · python/dns_query.py`	DNS queries built byte by byte over UDP
`linux/nat_lab.sh up | down`	namespaces lan, router (NAT), server
`python/stun_client.py`	your real public mapping

Key idea Everything this week runs on your machine first — then you look at real servers on the Internet.

SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · UDP (User Datagram Protocol) · DNS (Domain Name System) · NAT (network address translation) · CR (carriage return) · LF (line feed)

Task 1 · SMTP by hand

```
EHLO client.lab.test
MAIL FROM:<alice@lab.test>
RCPT TO:<bob@lab.test>
RCPT TO:<carol@lab.test>
DATA
From: Alice <alice@lab.test>
To: Bob <bob@lab.test>
Subject: typed by hand

Hello Bob,
..this line started with one dot.
.
QUIT
```

```
<- 220 localhost Python SMTP 1.4.6
<- 250-localhost
<- 250-8BITMIME
<- 250 HELP
<- 250 OK  (×3)
<- 354 End data with <CR><LF>.<CR><LF>
<- 250 OK
<- 221 Bye
```

```
To: Bob <bob@lab.test>
Subject: typed by hand
X-Peer: ('127.0.0.1', 41022)
X-MailFrom: alice@lab.test
X-RcptTo: bob@lab.test, carol@lab.test

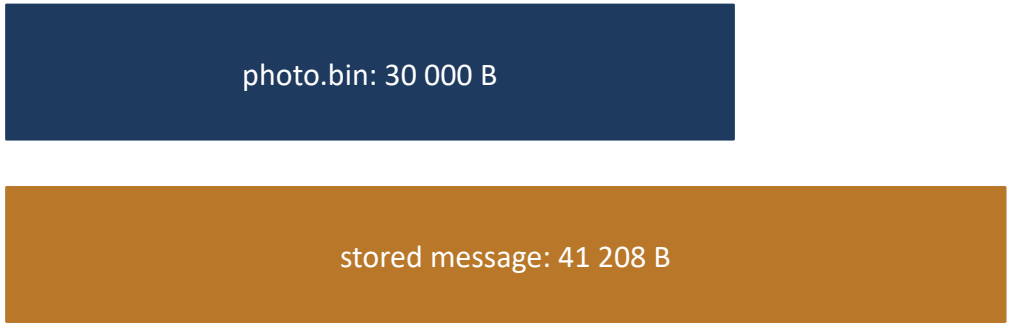
.this line started with one dot.
```

Key idea Q1–Q3 · Carol receives it from the envelope (X-RcptTo); the stored line has one dot.

SMTP (Simple Mail Transfer Protocol) · EHLO (extended hello) · RCPT (recipient) · CR (carriage return) · LF (line feed)

Task 2 · An attachment on the wire

```
head -c 30000 /dev/urandom > photo.bin
python3 python/send_mail.py 127.0.0.1 8025 photo.bin
ls -l photo.bin ~/Maildir/new/
less "$(ls -t ~/Maildir/new/* | head -1)"
```



+37 % — as the lecture predicted

```
Subject: Lab 5 report =?utf-8?b?4oCU?= first draft     →  4oCU = the dash "—" in UTF-8
```

1 Q4–Q5 · Find the envelope in the debug output, multipart/mixed with its boundary, and the attachment's base64 encoding.

Key idea MIME made visible: labels, boundaries, base64 and an encoded word.

MIME (Multipurpose Internet Mail Extensions) · UTF (Unicode Transformation Format)

Task 3 · POP3 by hand; an IMAP greeting over TLS

```
USER bob          → +OK send PASS
PASS secret       → +OK
STAT              → +OK 2 41423
LIST              → 1 215 / 2 41208 / .
UIDL
TOP 1 0
RETR 1            → … .
DELE 1
RSET              → unmarked
DELE 1
QUIT              → +OK bye (1 deleted)
```

```
python3 python/talk.py imap.gmail.com 993 --tls
<- * OK Gimap ready …
a1 CAPABILITY
<- * CAPABILITY IMAP4rev1 … IDLE … AUTH=…
<- a1 OK Thats all she wrote!
```

1 Q6 · No QUIT → no UPDATE state → nothing deleted.

2 Q7 · Tagged (a1) and untagged (*) lines; tcpdump sees only TLS records.

Key idea POP3 by hand shows its three states; the IMAP greeting shows a much larger protocol behind TLS.

POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · UIDL (unique-id listing) · STAT (status) · RETR (retrieve) · RSET (reset) · TLS (Transport Layer Security)

Task 4 · DNS with dig

```
dig www.unibuc.ro
dig +short unibuc.ro MX ; dig +short unibuc.ro NS
dig +trace www.unibuc.ro
dig +norecurse @a.root-servers.net www.unibuc.ro
dig @1.1.1.1 www.google.com A     # twice: TTL
dig gmail.com TXT ; dig _dmarc.gmail.com TXT
dig -x 8.8.8.8
dig +dnssec unibuc.ro SOA
curl -s -H 'accept: application/dns-json' \
  'https://cloudflare-dns.com/dns-query?name=unibuc.ro&type=MX'
```

header line
qr rd ra (aa?) · status

+trace
. → ro. → unibuc.ro.

+norecurse at a root
AUTHORITY: ro. NS … · ADDITIONAL: glue

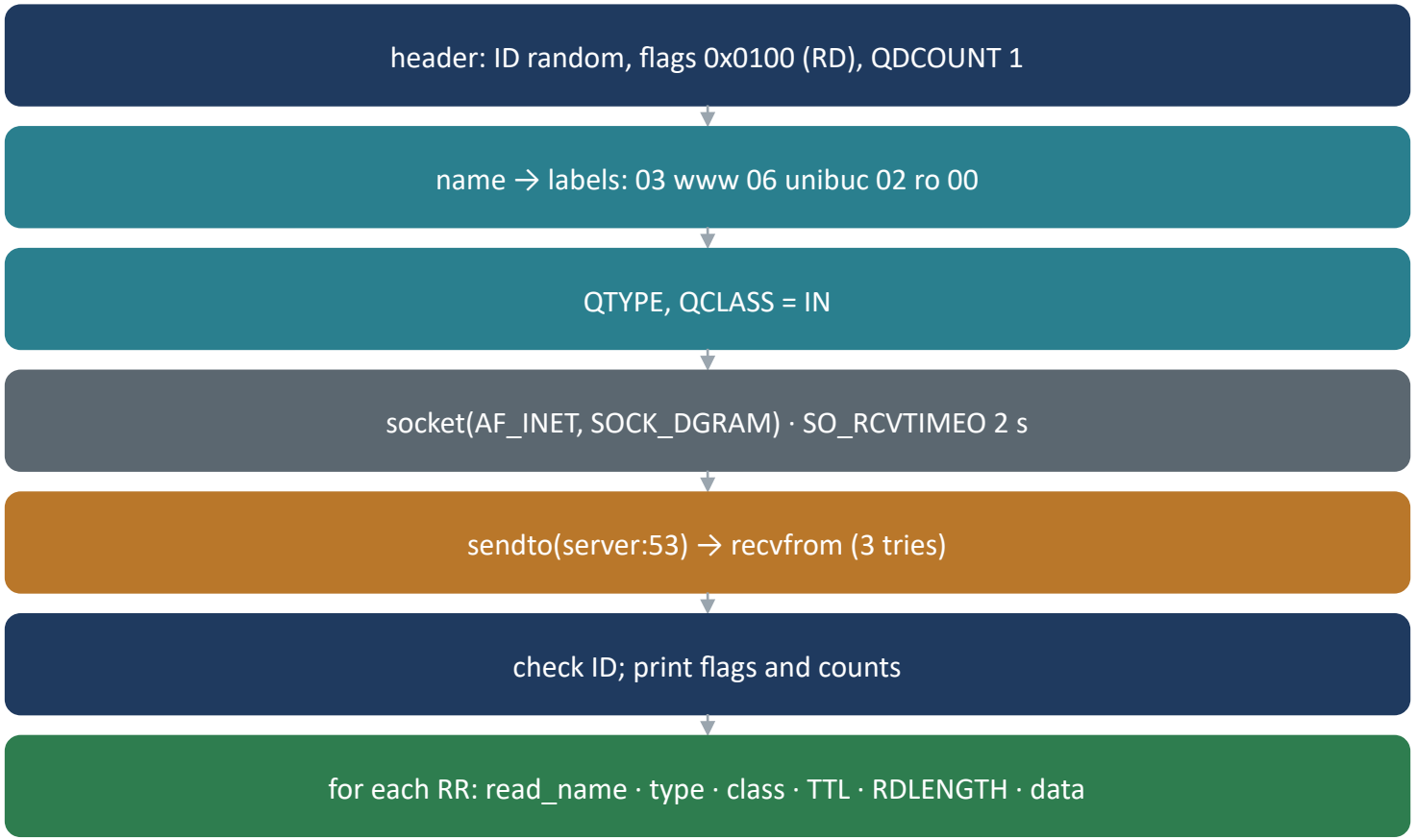
TTL
300 … then 287

DoH
invisible to tcpdump port 53

Key idea Q8–Q10 · dig is the stethoscope of DNS: flags, sections, referrals and TTLs at a glance.

DNS (Domain Name System) · SOA (start of authority) · TXT (text) · TTL (time to live) · DoH (DNS over HTTPS) · MX (mail exchanger) · NS (name server)

Task 5 · A DNS client in C: what the program does




```
make -C linux
./linux/dns_query \
   www.unibuc.ro A 1.1.1.1
./linux/dns_query unibuc.ro MX
./linux/dns_query \
   www.unibuc.ro A 198.41.0.4
```

```
sudo tcpdump -i any -n -X
      udp port 53
```

Key idea Q11 · Compare the bytes tcpdump prints with your hand-written homework query.

AF_INET (address family Internet) · DNS (Domain Name System) · TTL (time to live) · MX (mail exchanger) · RR (resource record) · RD (recursion desired) · IN (Internet class)

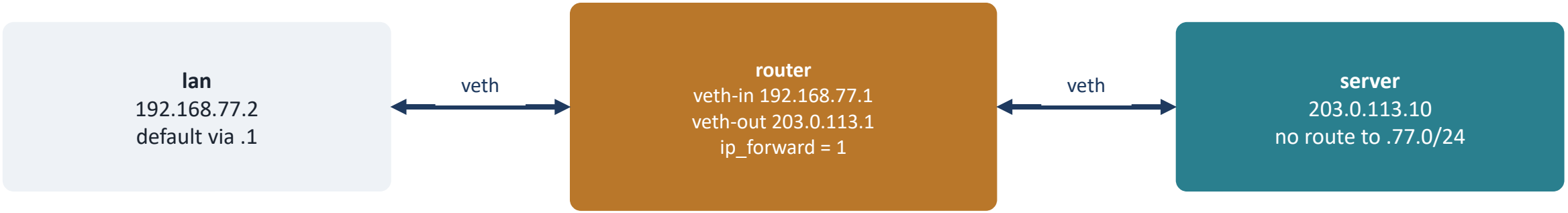
Task 5 · read_name: labels, pointers and a loop guard

```
for (;;) {
    if (p >= len || hops++ > 50) return -1; /* guard */
    int l = msg[p];
    if ((l & 0xC0) == 0xC0) {           /* pointer */
        if (!jumped) *pos = p + 2;
        p = ((l & 0x3F) << 8) | msg[p + 1];
        jumped = 1; continue;
    }
    if (l == 0) { if (!jumped) *pos = p + 1; break; }
    memcpy(out + n, msg + p + 1, l);    /* a label */
    n += l; out[n++] = '.'; p += 1 + l;
}
```

1. Q12 · `c0 0c` → offset 12, the question name.
2. After a pointer, parsing continues **after the 2 bytes**, not where the pointer led.
3. Q13 · Root replies have ANCOUNT 0: extend the program into an iterative resolver.

Key idea Never trust a reply: bounds checks and a jump limit turn a parser into a safe parser.

Task 6 · A LAN, a NAT router and a server in one machine




```
table ip nat {
  chain postrouting { type nat hook postrouting priority srcnat;
    oifname "veth-out" ip saddr 192.168.77.0/24 masquerade } }
```

```
sudo ./linux/nat_lab.sh up
sudo ip netns exec lan \
  curl -sI http://203.0.113.10/
```

1

Q14 · The server logs **203.0.113.1**, not 192.168.77.2; `conntrack -L` in the router shows both directions of the mapping.

Key idea Three namespaces, two cables, one rule: a complete NAT you can switch off and watch fail.

NAT (network address translation) · LAN (local area network)

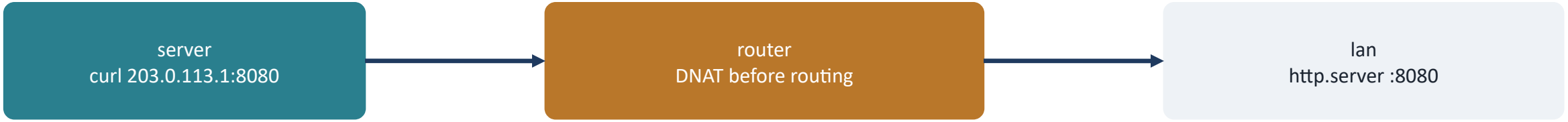
Task 6 · Remove NAT, then forward a port

`nft flush table ip nat`

the SYN reaches the server with source 192.168.77.2;

the SYN-ACK has no route back → **curl times out**

```
nft add chain ip nat prerouting \
 '{ type nat hook prerouting priority dstnat; … }'
nft add rule ip nat prerouting \
 iifname veth-out tcp dport 8080 \
 dnat to 192.168.77.2:8080
```



1 Q15 · DNAT in **prerouting** (the routing decision must see the new destination); masquerade in **postrouting** (after the outgoing interface is known). The lan server logs the real client 203.0.113.10.

Key idea Source NAT on the way out, destination NAT on the way in — the two halves of every home router.

SYN (synchronize) · ACK (acknowledgement) · NAT (network address translation)

Task 6 · STUN: which mapping did your real NAT create?

```
$ ip -4 addr show | grep inet
    inet 192.168.1.34/24 …
$ python3 python/stun_client.py
local (private) side: ('0.0.0.0', 52811)
request 1: the server saw us as 86.120.4.7:52811
request 2: the server saw us as 86.120.4.7:52811
```

private address: 192.168.1.34

public mapping: 86.120.4.7:52811

same port twice: endpoint-independent: hole punching works

Binding request: type 0x0001, magic cookie 0x2112A442, 12-byte transaction ID → response 0x0101 with **XOR-MAPPED-ADDRESS** (port XOR 0x2112, address XOR the cookie)

1 Q16 · Is your router's WAN address in 100.64.0.0/10? Then a CGNAT sits between you and the STUN server. (Example values: yours differ.)

Key idea STUN is a mirror on the Internet: it shows you the address your NAT gave you.

CGNAT (carrier-grade NAT) · STUN (Session Traversal Utilities for NAT) · NAT (network address translation) · WAN (wide area network)

Part D · Laboratory 5 on Windows

A · Lecture 5
Mail, news, names, NAT

→

B · Seminar 5
Problems with solutions

→

C · Laboratory 5 · Linux
SMTP, POP3, dig, DNS in C, NAT

→

D · Laboratory 5 · Windows
PowerShell, Winsock, your NAT

In this part

Tasks 1–3 · SMTP, MIME, POP3 and IMAP

Task 4 · Resolve-DnsName, nslookup, the cache

Task 5 · The DNS client with Winsock

Task 6 · Finding your own NAT

WSL 2 · Windows' own NAT

SMTP (Simple Mail Transfer Protocol) · MIME (Multipurpose Internet Mail Extensions) · POP3 (Post Office Protocol version 3) · IMAP (Internet Message Access Protocol) · DNS (Domain Name System) · NAT (network address translation) · WSL (Windows Subsystem for Linux)

Tasks 1–3 · The same mail servers on Windows

```
py -m pip install aiosmtpd
py -m aiosmtpd -n -l 127.0.0.1:8025 `
   -c aiosmtpd.handlers.Mailbox Maildir
py python\talk.py 127.0.0.1 8025
py python\send_mail.py 127.0.0.1 8025 photo.bin
py python\pop3_server.py Maildir 1110
py python\talk.py imap.gmail.com 993 --tls
```

random file
(New-Object Random).NextBytes($b)
[IO.File]::WriteAllBytes(…)

decode the encoded word
[Text.Encoding]::UTF8.GetString(
[Convert]::FromBase64String('4oCU'))

1 Wireshark on the loopback adapter, filter `tcp.port == 1110`: the POP3 password is readable — `tcp.port == 993`: only TLS, with the name in the ClientHello.

Key idea Q1–Q7 · Same protocols, same answers as on Linux — only the shell around them changes.

POP3 (Post Office Protocol version 3) · TLS (Transport Layer Security)

Task 4 · DNS with Windows tools

```
Resolve-DnsName www.unibuc.ro
Resolve-DnsName unibuc.ro -Type MX
Resolve-DnsName gmail.com -Type TXT -Server 1.1.1.1
Resolve-DnsName _dmarc.gmail.com -Type TXT
Resolve-DnsName 8.8.8.8                # PTR
Resolve-DnsName www.unibuc.ro `
   -Server a.root-servers.net -NoRecursion
nslookup -debug www.google.com 1.1.1.1
ipconfig /displaydns | Select-String …
ipconfig /flushdns                     # (admin)
Get-DnsClientServerAddress -AddressFamily IPv4
```

your resolvers
Get-DnsClientServerAddress

referral
-NoRecursion to a root server

TTL
nslookup -debug, twice

Windows cache
ipconfig /displaydns · /flushdns

Key idea Q8–Q10 · The cache you flush with ipconfig is a stub resolver’s cache — the resolver keeps its own.

IPv4 (Internet Protocol version 4) · DNS (Domain Name System) · PTR (pointer) · TXT (text) · TTL (time to live) · MX (mail exchanger)

Task 5 · The DNS client with Winsock

Linux	Windows
`(nothing)`	`WSAStartup(MAKEWORD(2, 2), &wsa)`
`int fd`	`SOCKET fd`
`struct timeval tv = {2, 0}`	`DWORD tv = 2000   (milliseconds)`
`close(fd)`	`closesocket(fd); WSACleanup()`
`gcc dns_query.c`	`gcc … -lws2_32 · cl … ws2_32.lib`

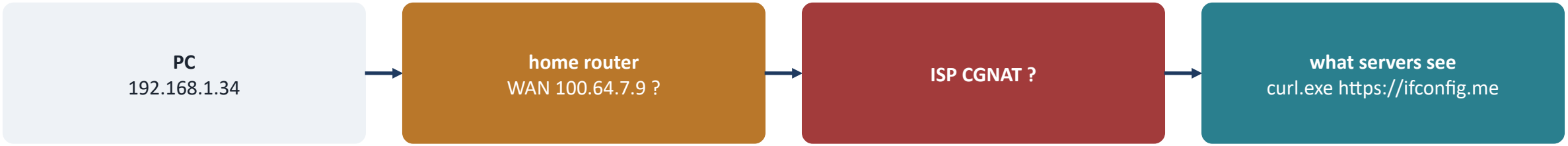
```
.\dns_query.exe www.unibuc.ro A 1.1.1.1
.\dns_query.exe www.unibuc.ro A 198.41.0.4
```

Wireshark filter **dns**
compare its tree with your output

Key idea Q11–Q13 · Wireshark decodes the same fields your program decodes — use it to check every byte.

WSAStartup (starts Windows Sockets) · WSACleanup (stops Windows Sockets) · DNS (Domain Name System) · fd (file descriptor)

Task 6 · How many NATs between you and the Internet?




```
ipconfig | Select-String 'IPv4|Gateway'
curl.exe -s https://ifconfig.me
tracert -d -h 4 1.1.1.1
py python\stun_client.py stun.cloudflare.com 3478
```

router WAN private or in **100.64.0.0/10**
→ two NATs on the path

1 Q14–Q15 · Count the translations; compare the STUN port with your local port and between two requests.

Key idea The address your PC knows is rarely the address the Internet sees.

CGNAT (carrier-grade NAT) · IPv4 (Internet Protocol version 4) · STUN (Session Traversal Utilities for NAT) · ISP (Internet service provider) · WAN (wide area network)

WSL 2 · Windows runs a NAT too



From another PC on the LAN, a server in WSL is unreachable without a port forwarding rule — Windows calls it netsh interface portproxy.

1 Q16 · Draw the packet from WSL to ifconfig.me with every address and every translation.

Key idea NAT is everywhere: in your router, at your ISP, and inside your own laptop.

NAT (network address translation) · ISP (Internet service provider) · LAN (local area network) · WSL (Windows Subsystem for Linux)

Next week

Lecture 6

multiplexing and demultiplexing with ports · UDP · the principles of reliable data transfer that TCP is built on

Seminar 6

checksums, sequence numbers, stop-and-wait and pipelined protocols

Laboratory 6

a reliable transfer protocol of your own over UDP, recovering from lost packets

Key idea The application layer is complete: every protocol of Weeks 3–5 rides on the transport services we open next.

TCP (Transmission Control Protocol) · UDP (User Datagram Protocol)